

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

ANAGHA A PAI (1BM24CS038)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING**

**in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)**

BENGALURU-560019

February 2026 to June 2026

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by ANAGHA A PAI (1BM24CS038), who is a bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Kanchana M Dixit
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index

Sl. No.	Experiment Title	Page No.
1	Write a program to obtain the Topological ordering of vertices in a given digraph.	8 - 9
2	Implement Johnson Trotter algorithm to generate permutations.	12 - 13
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	14 - 16
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	17 - 19
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	20 - 21
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	22 - 23
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	24 - 25
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	27 - 28 29 - 30
9	Implement Fractional Knapsack using Greedy technique.	31 - 32
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	33 - 34
11	Implement "N-Queens Problem" using Backtracking.	35 - 36

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write a program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>

#define MAX 10

int adj[MAX][MAX], visited[MAX], stack[MAX];
int n, top = -1;

void dfs(int v) {
    visited[v] = 1;

    for (int i = 0; i < n; i++) {
        if (adj[v][i] && !visited[i])
            dfs(i);
    }

    stack[++top] = v;
}

void topoSort() {
    for (int i = 0; i < n; i++) {
        if (!visited[i])
            dfs(i);
    }

    printf("Topological Order:\n");
    for (int i = top; i >= 0; i--)
        printf("%d ", stack[i]);
}

int main() {
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++) {
        visited[i] = 0;
        for (int j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }
}
```

```

    }

    topoSort();
    return 0;
}

```

Output:

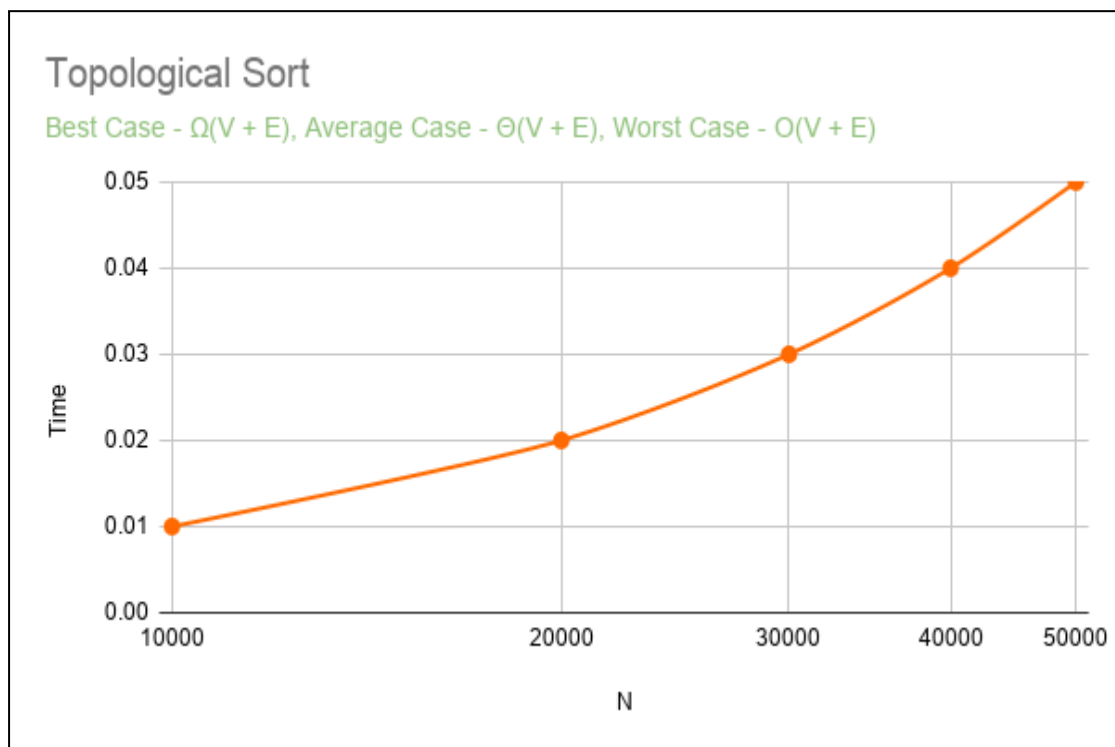
```

Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0
Topological Order:
0 2 1 3

=== Code Execution Successful ===

```

Graph:



Lab program 4: LEETCODE - Course Schedule

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you **must** take course b_i first if you want to take course a_i.

For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

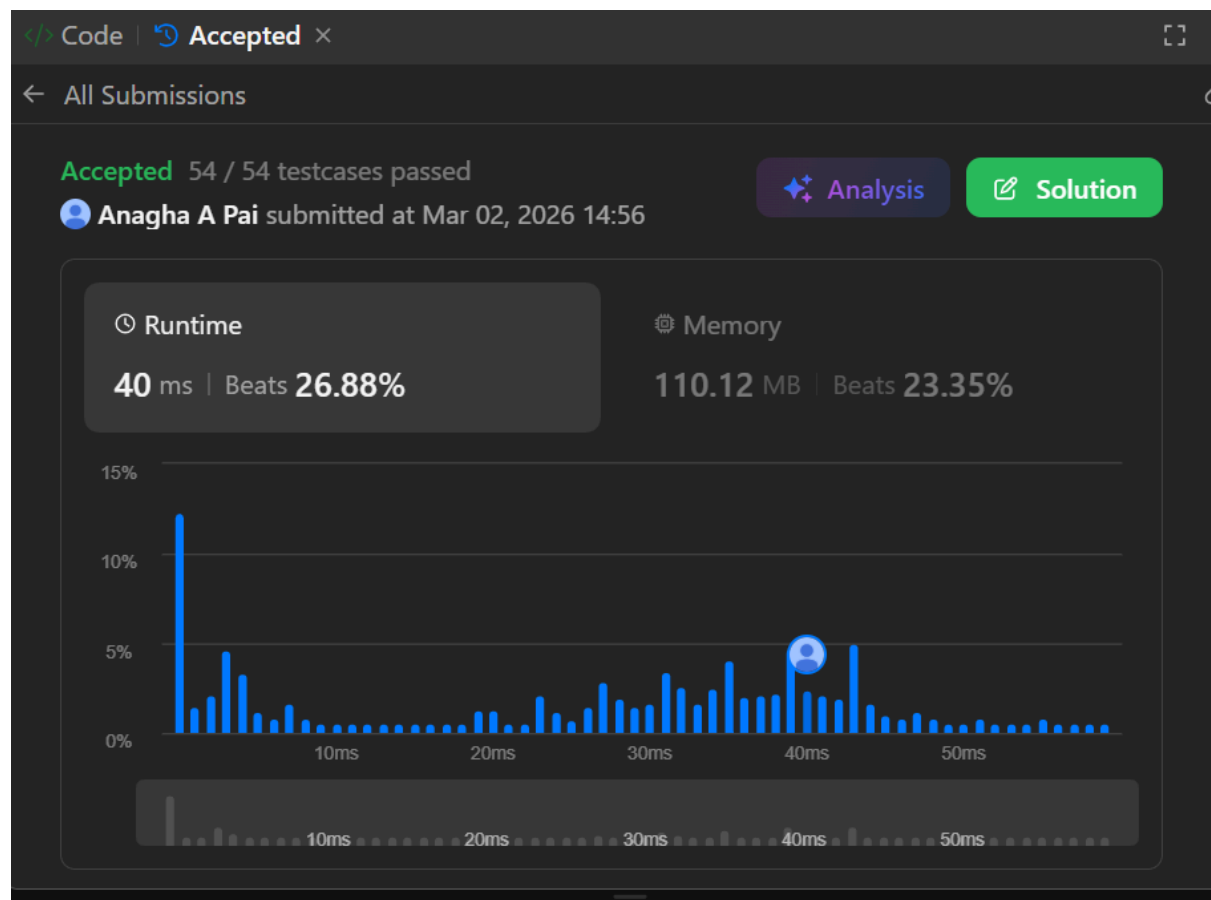
Return true if you can finish all courses. Otherwise, return false.

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize,
int* prerequisitesColSize) {
    int* indegree = (int*)calloc(numCourses, sizeof(int));
    int** graph = (int**)malloc(numCourses * sizeof(int*));
    int* graphSize = (int*)calloc(numCourses, sizeof(int));
    for (int i = 0; i < numCourses; i++) {
        graph[i] = (int*)malloc(prerequisitesSize * sizeof(int));
    }
    for (int i = 0; i < prerequisitesSize; i++) {
        int course = prerequisites[i][0];
        int prereq = prerequisites[i][1];

        graph[prereq][graphSize[prereq]++] = course;
        indegree[course]++;
    }
    int* queue = (int*)malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0)
            queue[rear++] = i;
    }
    int completed = 0;
    while (front < rear) {
        int curr = queue[front++];
        completed++;
        for (int i = 0; i < graphSize[curr]; i++) {
            int next = graph[curr][i];
            indegree[next]--;
            if (indegree[next] == 0)
                queue[rear++] = next;
        }
    }
    free(indegree);
    free(queue);
}
```

```
free(graphSize);  
for (int i = 0; i < numCourses; i++)  
    free(graph[i]);  
free(graph);  
return completed == numCourses;  
}
```

Output:



Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>

#define LEFT -1
#define RIGHT 1

void printPermutation(int perm[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", perm[i]);
    printf("\n");
}

int getMobile(int perm[], int dir[], int n) {
    int mobile = 0, mobile_index = -1;

    for (int i = 0; i < n; i++) {
        if (dir[perm[i] - 1] == LEFT && i != 0) {
            if (perm[i] > perm[i - 1] && perm[i] > mobile) {
                mobile = perm[i];
                mobile_index = i;
            }
        }

        if (dir[perm[i] - 1] == RIGHT && i != n - 1) {
            if (perm[i] > perm[i + 1] && perm[i] > mobile) {
                mobile = perm[i];
                mobile_index = i;
            }
        }
    }

    return mobile_index;
}

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```



```

void johnsonTrotter(int n) {
    int perm[n], dir[n];

    for (int i = 0; i < n; i++) {
        perm[i] = i + 1;
        dir[i] = LEFT;
    }

    printPermutation(perm, n);

    while (1) {
        int mobile_index = getMobile(perm, dir, n);

        if (mobile_index == -1)
            break;

        int mobile = perm[mobile_index];

        // Swap in its direction
        if (dir[mobile - 1] == LEFT)
            swap(&perm[mobile_index], &perm[mobile_index - 1]);
        else
            swap(&perm[mobile_index], &perm[mobile_index + 1]);

        int new_index;
        for (int i = 0; i < n; i++) {
            if (perm[i] == mobile) {
                new_index = i;
                break;
            }
        }

        for (int i = 0; i < n; i++) {
            if (perm[i] > mobile)
                dir[perm[i] - 1] *= -1;
        }

        printPermutation(perm, n);
    }
}

```

```
}

int main() {
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    johnsonTrotter(n);

    return 0;
}
```

Output:

```
Enter number of elements: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

=== Code Execution Successful ===
```

Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include <stdio.h>
#include <stdlib.h>
void merge(int arr[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];

    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    i = 0; j = 0; k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    while (i < n1) {
        arr[k++] = L[i++];
    }

    while (j < n2) {
        arr[k++] = R[j++];
    }
}
void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
```

```

        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

        merge(arr, l, m, r);
    }
}

int main() {
    int n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int *arr = (int *)malloc(n * sizeof(int));

    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    mergeSort(arr, 0, n - 1);

    printf("Sorted array:\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);

    free(arr);
    return 0;
}

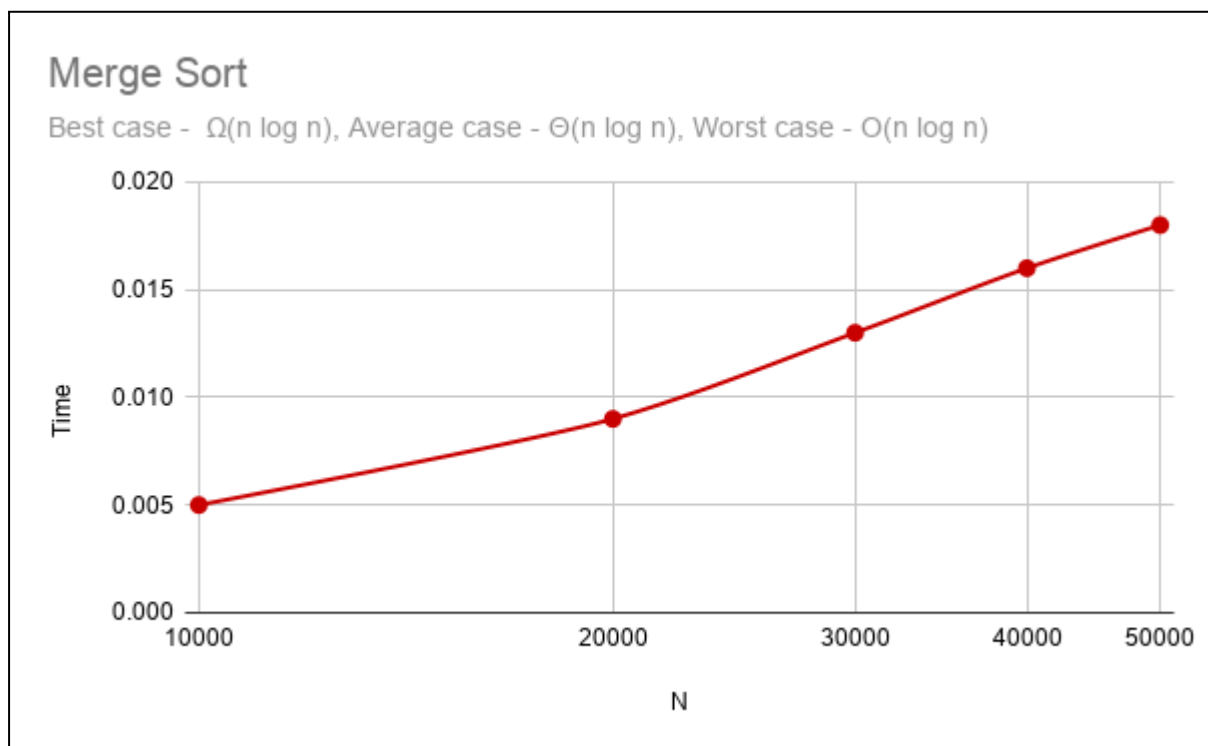
```

Output:

```
Default: "C:\Users\BMSCESCE-L3-25\Desktop
\anagha-1BM24CS038\merge sort\mergesort.exe"
ctrl+alt+1

"C:\Users\BMSCESCE-L3-25\D" x + v
Enter number of elements: 6
Enter elements:
34 23 21 89 76 54
Sorted array:
21 23 34 54 76 89
Process returned 0 (0x0)   execution time : 10.231 s
Press any key to continue.
|
```

Graph:



Lab program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include <stdio.h>

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[low]; // first element as pivot
    int i = low + 1;
    int j = high;

    while (1) {
        while (i <= high && arr[i] <= pivot)
            i++;

        while (arr[j] > pivot)
            j--;

        if (i >= j)
            break;

        swap(&arr[i], &arr[j]);
    }

    swap(&arr[low], &arr[j]);
    return j;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int p = partition(arr, low, high);
        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}
```

```

    }
}

int main() {
    int n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    quickSort(arr, 0, n - 1);

    printf("Sorted elements:\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}

```

Output:

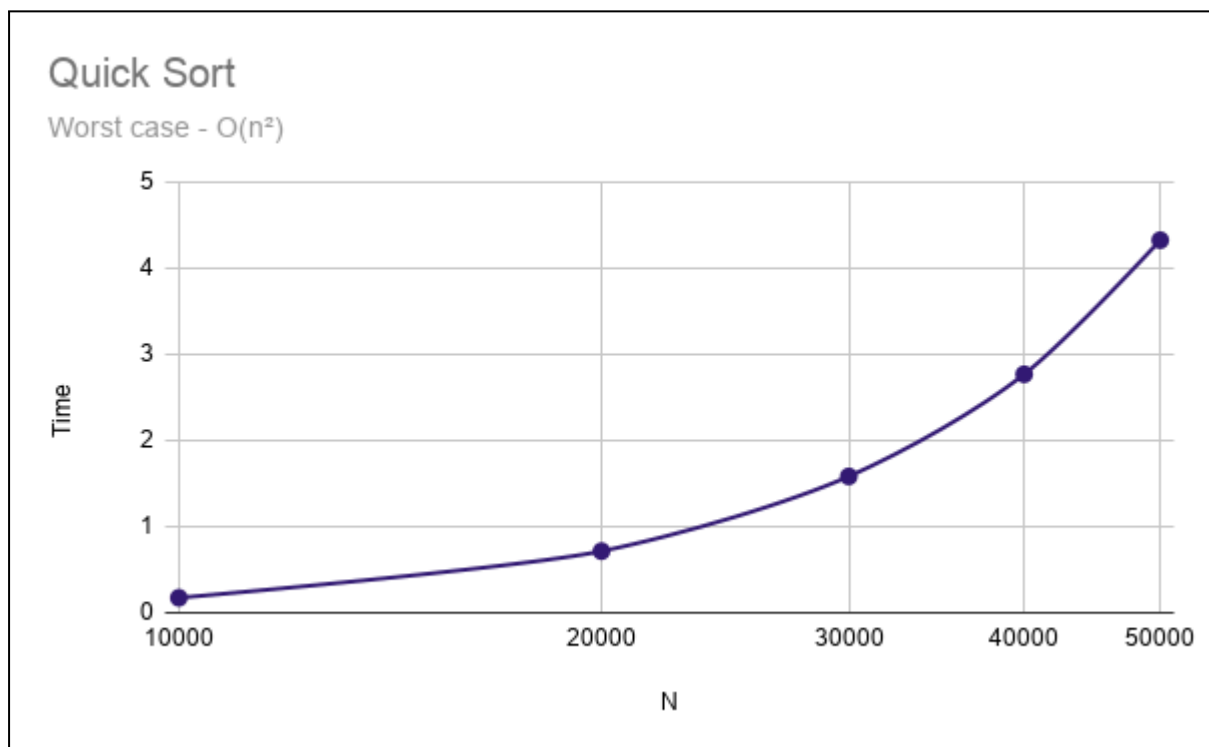
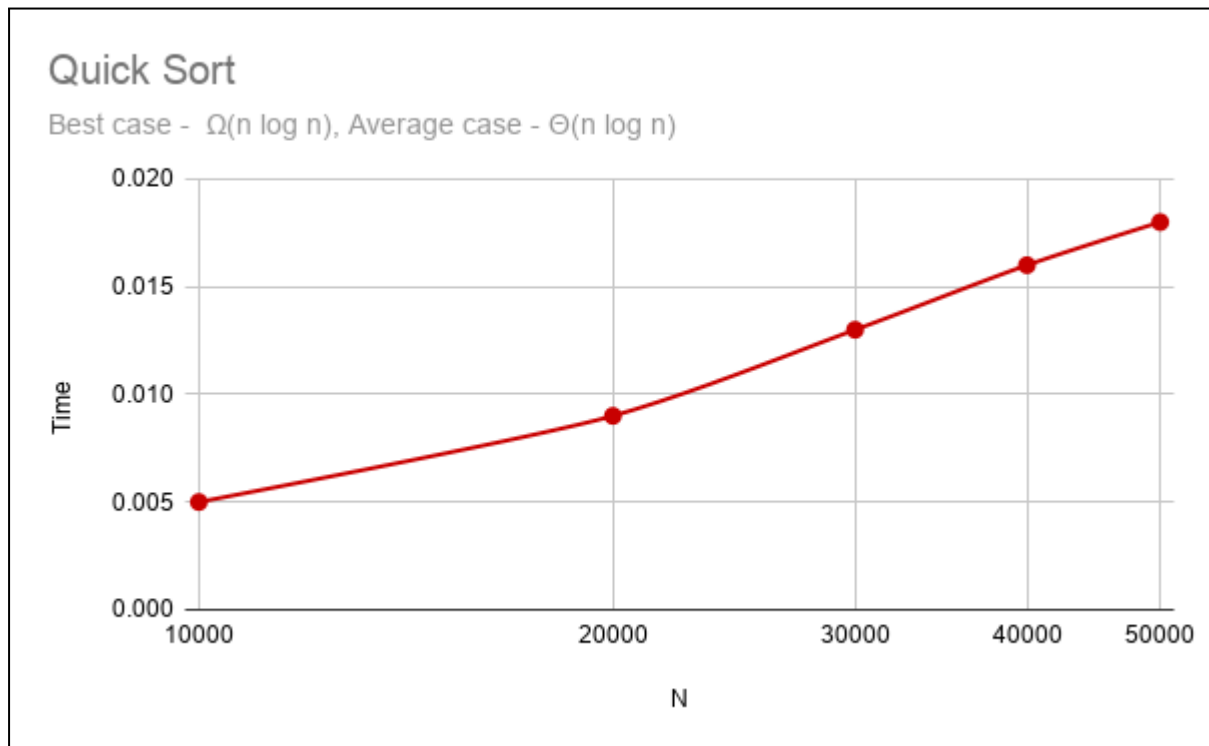
```

Enter number of elements: 6
Enter 6 elements:
45 34 23 67 56 76
Sorted elements:
23 34 45 56 67 76

=== Code Execution Successful ===

```

Graph:



Lab program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include<stdio.h>
```

```
void heapify(int a[], int n, int p)
```

```
{
    int c, item;
    item = a[p];
    c = 2 * p + 1;
    while(c < n)
    {

        if(c + 1 < n)
        {
            if(a[c] < a[c + 1])
            {
                c++;
            }
        }
        if(item < a[c])
        {
            a[p] = a[c];
            p = c;
            c = 2 * p + 1;
        }
        else
        {
            break;
        }
    }

    a[p] = item;
}
```

```
void build_heap(int a[], int n)
```

```
{
    int i;

    for(i = (n - 1) / 2; i >= 0; i--)
    {
```

```

        heapify(a, n, i);
    }
}

void heap_sort(int a[], int n)
{
    int i, temp;
    build_heap(a, n);

    for(i = n - 1; i > 0; i--)
    {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;

        heapify(a, i, 0);
    }
}

int main()
{
    int a[100], n;
    printf("Enter number of elements:\n");
    scanf("%d", &n);
    printf("Enter the array:\n");

    for(int i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    heap_sort(a, n);
    printf("Sorted array:\n");
    for(int i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }

    return 0;
}

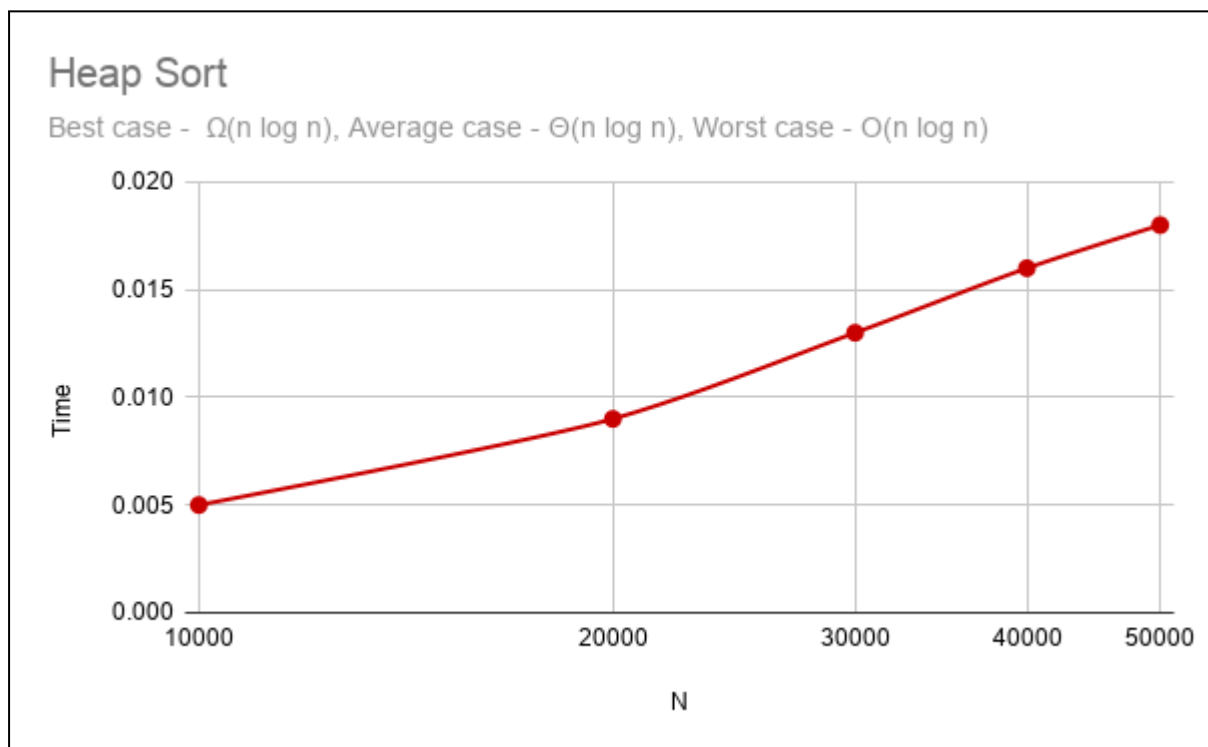
```

Output:

```
Default: "C:\Users\BMSCE-SCE-L3-25\Desktop
\anagha-1BM24CS038\Heap Sort\Heap Sort.exe"
ctrl+alt+1

"C:\Users\BMSCE-SCE-L3-25\D  X + v
Enter number of elements:
5
Enter the array:
23 56 3 32 54
Sorted array:
3 23 32 54 56
Process returned 0 (0x0)   execution time : 13.236 s
Press any key to continue.
```

Graph:



Lab program 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int knapsack(int W, int wt[], int val[], int n) {
    int dp[n+1][W+1];
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (wt[i-1] <= w)
                dp[i][w] = max(val[i-1] + dp[i-1][w - wt[i-1]],
                               dp[i-1][w]);
            else
                dp[i][w] = dp[i-1][w];
        }
    }
    return dp[n][W];
}

int main() {
    int n, W;
    printf("Enter number of items: ");
    scanf("%d", &n);
    int val[n], wt[n];
    printf("Enter values of items:\n");
    for (int i = 0; i < n; i++) {
```

```

        scanf("%d", &val[i]);
    }
    printf("Enter weights of items:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &wt[i]);
    }
    printf("Enter capacity of knapsack: ");
    scanf("%d", &W);
    printf("Maximum value = %d\n", knapsack(W, wt, val, n));
    return 0;
}

```

Output:

```

Enter number of items: 4
Enter values of items:
3 4 5 6
Enter weights of items:
1 2 3 5
Enter capacity of knapsack: 8
Maximum value = 13

=== Code Execution Successful ===

```

Lab program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include<stdio.h>
#define INF 9999

void floydWarshall(int graph[10][10], int n)
{
    int i, j, k;

    // Updating shortest distances
    for(k = 0; k < n; k++)
    {
        for(i = 0; i < n; i++)
        {
            for(j = 0; j < n; j++)
            {
                if(graph[i][k] + graph[k][j] < graph[i][j])
                {
                    graph[i][j] = graph[i][k] + graph[k][j];
                }
            }
        }
    }

    printf("Shortest distance matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            if(graph[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", graph[i][j]);
        }
        printf("\n");
    }
}

int main()
{
```

```

int n, i, j;
int graph[10][10];
printf("Enter number of vertices:\n");
scanf("%d", &n);
printf("Enter adjacency matrix:\n");
printf("Use %d for no direct edge\n", INF);
for(i = 0; i < n; i++)
{
    for(j = 0; j < n; j++)
    {
        scanf("%d", &graph[i][j]);
    }
}
floydWarshall(graph, n);
return 0;
}

```

Output:

```

Default: "C:\Users\BMSCE-SCE-L3-25\Desktop
\anagha-1BM24CS038\floyds algoritim\bin\Debug
\floyds algoritim.exe"
ctrl+alt+1

Enter number of vertices:
4
Enter adjacency matrix:
Use 9999 for no direct edge
0 5 9999 10
9999 0 3 9999
9999 9999 0 1
9999 9999 9999 0
Shortest distance matrix:
0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0

Process returned 0 (0x0)   execution time : 78.174 s
Press any key to continue.

```

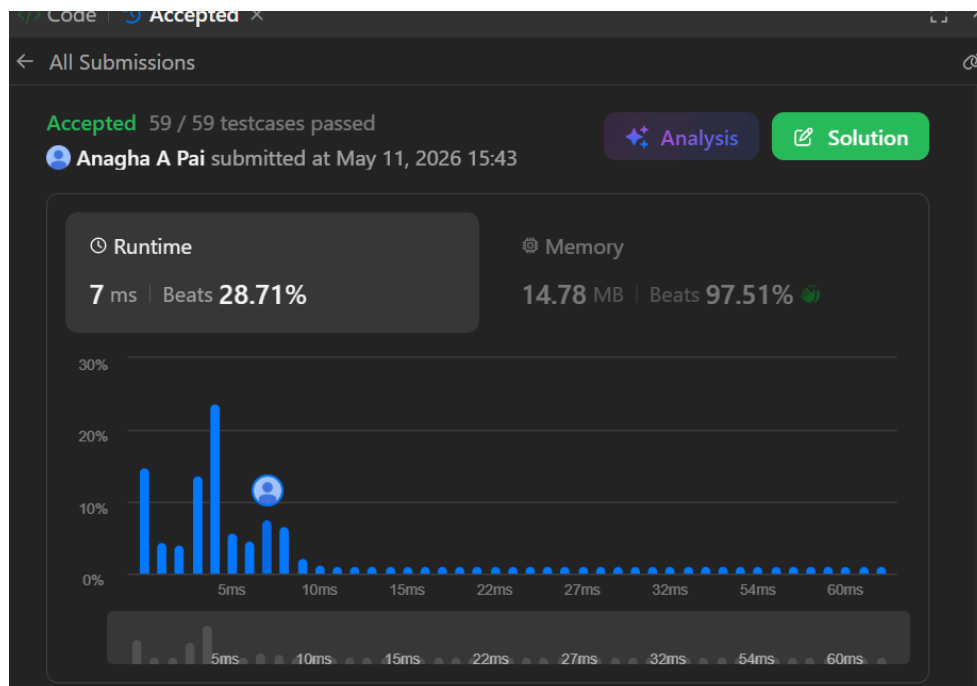
Lab program 10: LEETCODE - Find the duplicate number

Given an array of integers `nums` containing $n + 1$ integers where each integer is in the range $[1, n]$ inclusive.

There is only **one repeated number** in `nums`, return *this repeated number*.

```
int findDuplicate(int* nums, int numsSize) {  
    int slow = nums[0];  
    int fast = nums[0];  
    do {  
        slow = nums[slow];  
        fast = nums[nums[fast]];  
    } while (slow != fast);  
    slow = nums[0];  
    while (slow != fast) {  
        slow = nums[slow];  
        fast = nums[fast];  
    }  
    return slow;  
}
```

Output:



Lab program 8a:

Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>
#include <limits.h>

#define m 100

int V;

int minKey(int key[], int mstSet[]) {
    int min = INT_MAX, min_index;

    for (int v = 0; v < V; v++) {
        if (mstSet[v] == 0 && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}

void printMST(int parent[], int graph[m][m]) {
    int cost = 0;
    printf("\nEdge \tWeight\n");
    for (int i = 1; i < V; i++) {
        printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);
        cost += graph[i][parent[i]];
    }
    printf("Total Cost = %d\n", cost);
}

void primMST(int graph[m][m]) {
    int parent[m], key[m], mstSet[m];

    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }
}
```

```

key[0] = 0;
parent[0] = -1;

for (int count = 0; count < V - 1; count++) {
    int u = minKey(key, mstSet);
    mstSet[u] = 1;

    for (int v = 0; v < V; v++) {
        if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
            parent[v] = u;
            key[v] = graph[u][v];
        }
    }
}

printMST(parent, graph);
}

int main() {
    int graph[m][m];

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter adjacency matrix (0 if no edge):\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    primMST(graph);

    return 0;
}

```

Output:

```
Default: "C:\Users\BMSCE-SCE-L3-25\Desktop
\anagha-1BM24CS038\Prims Algorithm\Prims
Algorithm.exe"
ctrl+alt+1

"C:\Users\BMSCE-SCE-L3-25\D  X + v

Enter number of vertices: 4
Enter adjacency matrix (0 if no edge):
0 1 0 1
0 0 1 0
0 1 1 0
0 0 1 0

Edge    Weight
0 - 1    0
1 - 2    1
0 - 3    0
Total Cost = 1

Process returned 0 (0x0)    execution time : 21.534 s
Press any key to continue.
|
```

Lab program 8b:

Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include<stdio.h>
#include<string.h>
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

struct Edge {
    int src, dest, weight;
};

int parent[MAX];

int find(int i) {
    while (parent[i] != i)
        i = parent[i];
    return i;
}

void unionSet(int u, int v) {
    int u_root = find(u);
    int v_root = find(v);
    parent[u_root] = v_root;
}

int compare(const void* a, const void* b) {
    return ((struct Edge*)a)->weight - ((struct Edge*)b)->weight;
}

int main() {
    int V, E;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter number of edges: ");
    scanf("%d", &E);
```

```

struct Edge edges[MAX];

printf("Enter edges (src dest weight):\n");
for (int i = 0; i < E; i++) {
    scanf("%d %d %d", &edges[i].src, &edges[i].dest, &edges[i].weight);
}

for (int i = 0; i < V; i++)
    parent[i] = i;

qsort(edges, E, sizeof(edges[0]), compare);

int count = 0, i = 0, cost = 0;

printf("\nEdge \tWeight\n");

while (count < V - 1 && i < E) {
    struct Edge next = edges[i++];

    int x = find(next.src);
    int y = find(next.dest);

    if (x != y) {
        printf("%d - %d \t%d\n", next.src, next.dest, next.weight);
        cost += next.weight;
        unionSet(x, y);
        count++;
    }
}

printf("Total Cost = %d\n", cost);

return 0;
}

```

Output:

```
Enter number of vertices: 4
Enter number of edges: 5
Enter edges (src dest weight):
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

Edge    Weight
2 - 3    4
0 - 3    5
0 - 1   10
Total Cost = 19

=== Code Execution Successful ===
```

Lab program 9:

Implement fractional knapsack problem using Greedy technique.

```
#include<stdio.h>
void main()
{

    float p[100],w[100],x[100],n,maxcap,total=0.0,temp1,temp2;
    printf("enter the number of elements\n");
    scanf("%f",&n);
    float rc=0;
    printf("Enter the values\n");
    for(int i=0;i<n;i++)
    {
        scanf("%f",&p[i]);
        scanf("%f",&w[i]);
        x[i]=0.0;
    }
    printf("enter the maximum capacity\n");
    scanf("%f",&maxcap);
    rc=maxcap;
    for(int i=0;i<n;i++)
    {
        for(int k=0;k<n-1;k++)
        {
            if(p[i]/w[i]>p[k]/w[k])
            {
                temp1=p[i];
                p[i]=p[k];
                p[k]=temp1;
                temp2=w[i];
                w[i]=w[k];
                w[k]=temp2;
            }
        }
    }
    int j;
    for( j=0;j<n;j++)
    {

        if(w[j]<rc)
```

```

        {
            rc-=w[j];
            x[j]=1.0;
        }
        else
            break;
    }
    if(j<n)
    {
        x[j]=rc/w[j];
    }
    for(int i=0;i<n;i++)
    {
        total=total+(x[i]*p[i]);
    }
    printf("the total profit: %f",total);
}

```

Output:

```

enter the number of elements
5
Enter the values
4 5 6 7 3
6 7 4 5 3
enter the maximum capacity
8
the total profit: 12.857143

=== Code Exited With Errors ===

```


Lab program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>
#define INF 99999
int minDistance(int dist[], int visited[], int n) {
    int min = INF, min_index = -1;
    for (int v = 0; v < n; v++) {
        if (!visited[v] && dist[v] <= min) {
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
}
void dijkstra(int n, int graph[n][n], int src) {
    int dist[n], visited[n];

    for (int i = 0; i < n; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }

    dist[src] = 0;
    for (int count = 0; count < n - 1; count++) {
        int u = minDistance(dist, visited, n);
        if (u == -1) break;
        visited[u] = 1;
        for (int v = 0; v < n; v++) {
            if (!visited[v] && graph[u][v] &&
                dist[u] != INF &&
                dist[u] + graph[u][v] < dist[v]) {

                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("Vertex\tDistance from Source\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\n", i, dist[i]);
    }
}
```

```

    }
}

int main() {
    int n;
    printf("Enter the number of vertices of the graph:\n");
    scanf("%d", &n);

    int graph[n][n];

    printf("Enter the adjacency matrix values:\n");
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    dijkstra(n, graph, 0);
    return 0;
}

```

Output:

```

Enter the number of vertices of the graph:
5
Enter the adjacency matrix values:
0  10  0  30  100
0  0  50  0  0
0  0  0  0  10
0  0  20  0  60
0  0  0  0  0
Vertex  Distance from Source
0  0
1  10
2  50
3  30
4  60

```

Lab program 11:

Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>

#define MAX 20

int board[MAX][MAX];
int N;
int isSafe(int row, int col) {
    int i, j;

    // Check left side of current row
    for (i = 0; i < col; i++) {
        if (board[row][i])
            return 0;
    }
    for (i = row, j = col; i >= 0 && j >= 0; i--, j--) {
        if (board[i][j])
            return 0;
    }
    for (i = row, j = col; i < N && j >= 0; i++, j--) {
        if (board[i][j])
            return 0;
    }

    return 1;
}

int solveNQ(int col) {

    // If all queens are placed
    if (col >= N)
        return 1;

    int i;
    for (i = 0; i < N; i++) {

        // Check if safe
        if (isSafe(i, col)) {

            // Place queen
            board[i][col] = 1;
```

```

        if (solveNQ(col + 1))
            return 1;
        board[i][col] = 0;
    }
}

return 0;
}

void printBoard() {
    int i, j;

    printf("\nSolution Exists:\n\n");

    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {

            if (board[i][j])
                printf("Q ");
            else
                printf(". ");
        }
        printf("\n");
    }
}

int main() {

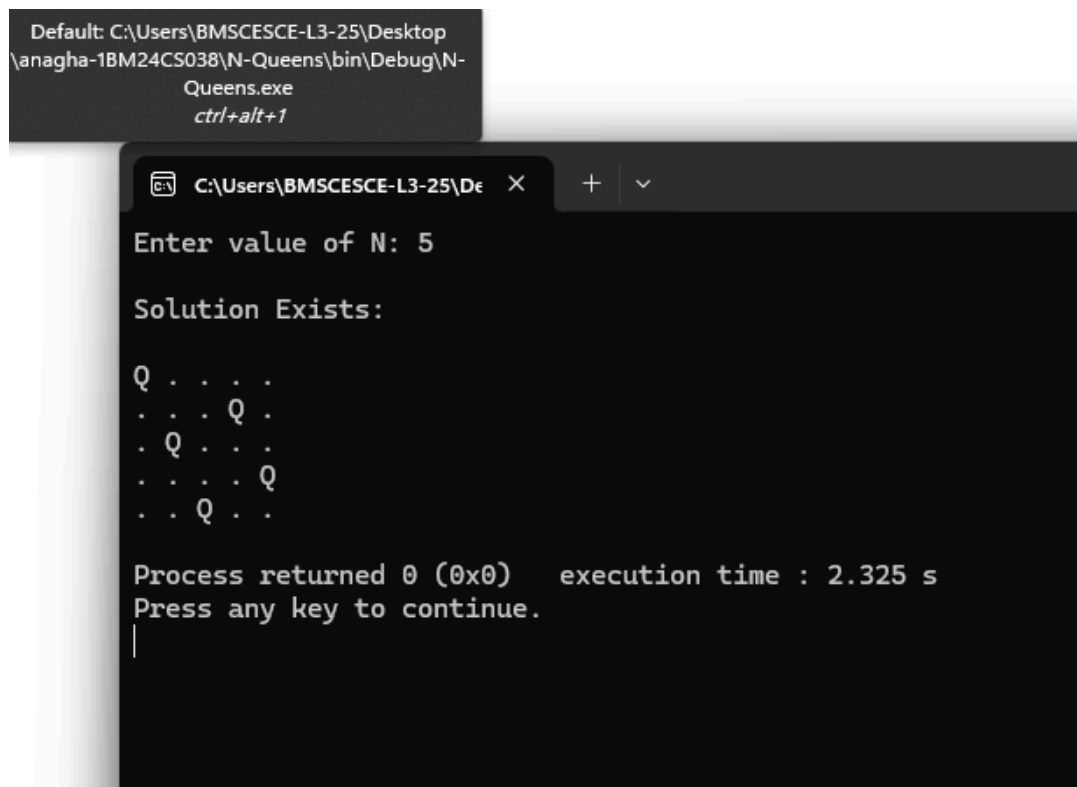
    printf("Enter value of N: ");
    scanf("%d", &N);

    if (solveNQ(0))
        printBoard();
    else
        printf("No Solution Exists");

    return 0;
}

```

Output:



```
Default: C:\Users\BMSCE-SCE-L3-25\Desktop
\nagha-1BM24CS038\N-Queens\bin\Debug\N-
Queens.exe
ctrl+alt+1

C:\Users\BMSCE-SCE-L3-25\Desktop >
Enter value of N: 5

Solution Exists:

Q . . . .
. . . Q .
. Q . . .
. . . . Q
. . Q . .

Process returned 0 (0x0)   execution time : 2.325 s
Press any key to continue.
|
```

Lab program : LEETCODE - Container with most water

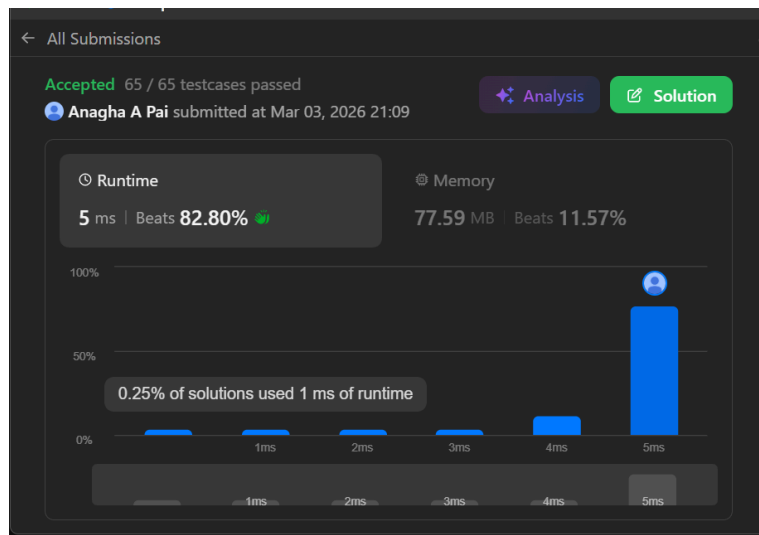
You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the i^{th} line are $(i, 0)$ and $(i, \text{height}[i])$.

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return *the maximum amount of water a container can store*.

```
int maxArea(int* height, int heightSize) {
    int left = 0;
    int right = heightSize - 1;
    int maxWater = 0;
    while (left < right) {
        int h = (height[left] < height[right]) ? height[left] :
height[right];
        int width = right - left;
        int area = h * width;
        if (area > maxWater)
            maxWater = area;
        if (height[left] < height[right])
            left++;
        else
            right--;
    }
    return maxWater;
}
```

Output:



Lab program : LEETCODE - First and Last position of element in a sorted array

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given target value.

If the target is not found in the array, return `[-1, -1]`.

```
int findFirst(int* nums, int numsSize, int target){
    int left = 0, right = numsSize - 1;
    int ans = -1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) {
            ans = mid;
            right = mid - 1;
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return ans;
}

int findLast(int* nums, int numsSize, int target) {
    int left = 0, right = numsSize - 1;
    int ans = -1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) {
            ans = mid;
        }
    }
}
```

```

        left = mid + 1;
    } else if (nums[mid] < target) {
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}

return ans;
}

int* searchRange(int* nums, int numsSize, int target, int* returnSize) {
    *returnSize = 2;
    int* result = (int*)malloc(2 * sizeof(int));
    result[0] = findFirst(nums, numsSize, target);
    result[1] = findLast(nums, numsSize, target);

    return result;
}

```

Output:

